

JavaScript 3: DOM Manipulation

ผศ. ดร. เกี๊จแกว ธเนศวร

kejkaew.tha@kmutt.ac.th

ตารางเรียน

สัปดาห์	วันที่	หัวข้อ/รายละเอียด	Assignment (ผู้ตรวจ)	ส่ง Assignment
1	4/8/2569	Introduction to Web Development & HTML5	Assignment 1 (KT,KN)	1
2	11/8/2569	CSS3 Fundamentals	Assignment 2 (KT, KN)	1,2
3	18/8/2569	Advanced CSS & UI Layouts	Assignment 3 (KT)	1,2,3
4	25/8/2569	Responsive Web Design	Assignment 4 (KT, KN)	2,3,4
5	1/9/2569	JavaScript 1: Variables, Data Types, DOM & Event Handling	Assignment 5 (KT, KN)	3,4,5
6	8/9/2569	JavaScript 2: Arrays, Objects & Logic Control	Assignment 6 (KT)	4,5,6
7	15/9/2569	JavaScript 3: DOM Manipulation	Assignment 7 (KT, KN)	5,6,7
8	22/9/2569	Asynchronous JS, AJAX & JSON Data Flow	Assignment 8 (KT, KN)	6,7,8
9	29/9/2569	Introduction to Node.js & Express Basics	Assignment 9 (KT)	7,8,9
10	6/10/2569	Backend Architecture & RESTful API (CRUD)	Assignment 10 (KT, KN)	8,9,10
	13/10/2569	หยุด		
	20/10/2569	หยุด		
	27/10/2569	หยุด		
11	3/11/2569	Microservices Architecture Concept & Hands-on	Assignment 11 (KT, KN)	9,10,11
12	10/11/2569	Database Integration (MySQL / Firebase)	Assignment 12 (KT, KN)	10,11,12
13	17/11/2569			11,12
14	24/11/2569	ส่งโปรเจก (9:00-16:00)	KT	12
15	12/2569	สอบปลายภาค ในตาราง		

ส่ง portfolio web page

- กำหนดส่งวันที่ 9 ต.ค. 2569 ก่อน 23:55 น.
- ส่งโดยอัปโหลด zip file ลง LEB2 และพิมพ์ URL link ของ website ของนศ (เช่น GitHub) ตอนส่งใน LEB2 ด้วย

ขั้นตอนการ zip file

1. นำ website files ทั้งหมดใส่ใน folder เดียวกัน
2. Compress file โดยใช้ ZIP compression
3. อัป zip file ลง LEB2 ที่กำหนดให้ พร้อมทั้งใส่ URL link ของ website ของเรา
4. ส่งโดยดูรายละเอียดตามหน้าถัดไป
5. อย่าลืม references (แหล่งอ้างอิงทั้งหลาย) ถ้าใครไปเอา source codes, template หรือ เนื้อหาบางอย่าง มาจากที่อื่น ควรใส่แหล่งอ้างอิง

- Portfolio web page (อยู่ใน index.html) ของนักศึกษาแต่ละคน มีรายละเอียดข้อมูลดังนี้
 - มีชื่อ - นามสกุล
 - งานอดิเรก
 - ทักษะ ความสามารถ
 - ความสนใจ
 - รูป
 - มี Link ไปเว็บอื่น เช่น Facebook, Instagram ของเรา
 - หรือรายละเอียดอื่น ๆ (ข้อมูลส่วนตัวเช่น เลขบัตรประชาชน ไม่เอานะ!)

- ใน project.html เนื้อหาของเพจเป็นเรื่องเกี่ยวกับ project ต่าง ๆ ที่เราเคยทำหรือคิดว่าจะทำ มีมากกว่า 1 อย่างก็ได้
 - เช่น senior project ที่เราจะทำ หรือ project จากที่เคยทำในห้องเรียน
 - ทำ link เชื่อมระหว่าง portfolio (index.html) กับ project (project.html) ด้วย
- HTML ทั้ง 2 files ใช้ CSS file อันเดียวกัน และ ทั้ง 2 HTML files ต้องมี CSS แบบ external CSS นะ

-
- ใน portfolio HTML file มี link ไปหา หน้า page ที่เราทำ assignment ในห้องเรียน ได้แก่
 - Cinnamon Roll Wreath with Apples
 - Pizza dip
 - Blog
 - Register and login
 - Square game
 - ทั้งหมดนี้ ต้องมี CSS และ JavaScript แบบ external นะคะ

คำแนะนำ

- การเรียก path ไปหารูป หรือ files ต่าง ๆ ให้ใช้แบบ relative file path โดยอ้างอิงจากตำแหน่ง HTML file นั้น
- ใช้ CSS หรือ JavaScript framework อะไรก็ได้
- **ไม่ให้ใช้ Content management system (CMS) หรือ website builder**
- **ควรทำเอง ใช้ AI ช่วยแค่เฉพาะบางจุดได้ จะได้เรียนรู้ด้วย**

เกณฑ์การให้คะแนน (15 คะแนน)

รายละเอียด	คะแนน
มีรายละเอียดงานต่าง ๆ ทั้งหมดครบถ้วน	5 คะแนน
แยก CSS และ JavaScript files ได้ถูกต้อง	5 คะแนน
Responsive design	3 คะแนน
สวยงาม (ขนาดตัวอักษร สี background etc.) และ user friendly	2 คะแนน
ไม่ใส่แหล่งอ้างอิง (ถ้าจำเป็นต้องใส่ เช่น Recipe หรือใช้ template จากที่อื่น)	-1 คะแนน/ต่อจุด
คำหยาบ ทุกประเภท + รูปที่ไม่เหมาะสม, ไม่ส่ง link URL / ไม่ส่ง zip file	ได้ 0 ทันที

เราเรียนอะไรแล้วบ้าง

- HTML
- CSS
- JavaScript
 - Variable, control flow, array, event handler
 - Form

วันนี้เราจะเรียนเรื่อง

- Timer
- Create new elements or nodes
- JavaScript events (object)
- สัปดาห์หน้า สอบ Quiz 5 คะแนน
 - เตรียมปากกา ดินสอ มาด้วย
 - ห้ามใช้ AI ทุกประเภท ห้ามคุยถามเพื่อนๆ

Setting a timer

method	description
<code>setTimeout(function,delayMS);</code>	ตั้งเวลาเพื่อเรียกใช้งานฟังก์ชันที่กำหนด เมื่อผ่านระยะเวลาที่ระบุไว้ (หน่วยเป็นมิลลิวินาที)
<code>setInterval(function,delayMS);</code>	ตั้งเวลาเพื่อเรียกใช้งานฟังก์ชันซ้ำๆ อย่างต่อเนื่องในทุกๆ ช่วงเวลาที่กำหนด (หน่วยเป็นมิลลิวินาที)
<code>clearTimeout(timerID);clearInterval(timerID);</code>	สั่งหยุดการทำงานของตัวจับเวลาตามที่ระบุไว้

- ทั้ง `setTimeout` และ `setInterval` ID จะ return ID สำหรับ timer
- เราสามารถใช้ ID ตัวนั้นในการ `clearTimeout/Interval` เพื่อหยุดเวลาได้

Timer: setTimeout

```
119 function delayedMessage() {  
120     const output = document.getElementById("output");  
121     output.textContent = "Wait for it...";  
122     setTimeout(sayHello, 5000);  
123 }  
124  
125 function sayHello() {  
126     const output = document.getElementById("output");  
127     output.textContent = "Hello!";  
128 }  
129
```

กดปุ่ม setTimeout แล้วรอ 5 วินาที ข้อความเปลี่ยนเป็น Hello!

Timer: setTimeout — ผลการรัน

1. Setting a Timer (setTimeout, setInterval, Parameters)

1.1 setTimeout

Hello!

Click me (setTimeout 5s)

กดปุ่ม setTimeout แล้วรอ 5 วินาที ข้อความเปลี่ยนเป็น Hello!

ตัวอย่าง

JS

```
131 let intervalTimer = null;
132
133 function toggleIntervalMDT() {
134     if (intervalTimer === null) {
135         intervalTimer = setInterval(showMDT, 1000);
136     } else {
137         clearInterval(intervalTimer);
138         intervalTimer = null;
139     }
140 }
141
142 function showMDT() {
143     const output = document.getElementById("outputInterval");
144     output.textContent += " MDT!";
145 }
```

Example

1.2 setInterval

MDT! MDT! MDT! MDT! MDT! MDT! MDT! MDT!

Start / Stop setInterval

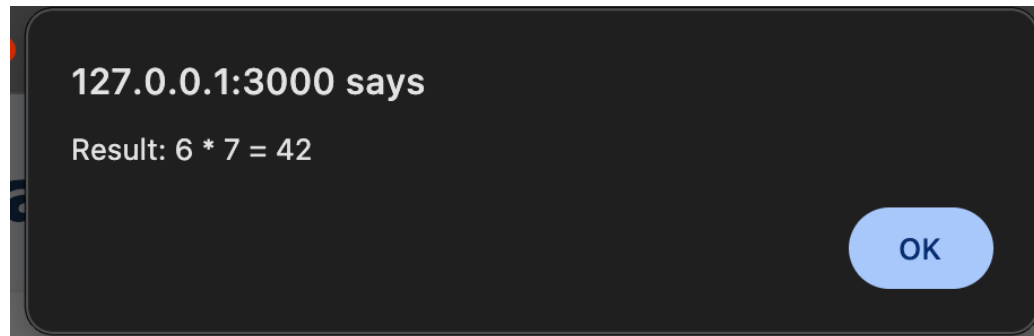
Passing parameters to timers

- เราสามารถส่ง parameter ผ่านทาง timer function ได้ โดยการเพิ่ม argument ใน timer function

JS

```
148 function delayedMultiply() {  
149     setTimeout(multiply, 2000, 6, 7);  
150 }  
151  
152 function multiply(a, b) {  
153     alert(`Result: ${a} * ${b} = ${a * b}`);  
154 }
```

Example



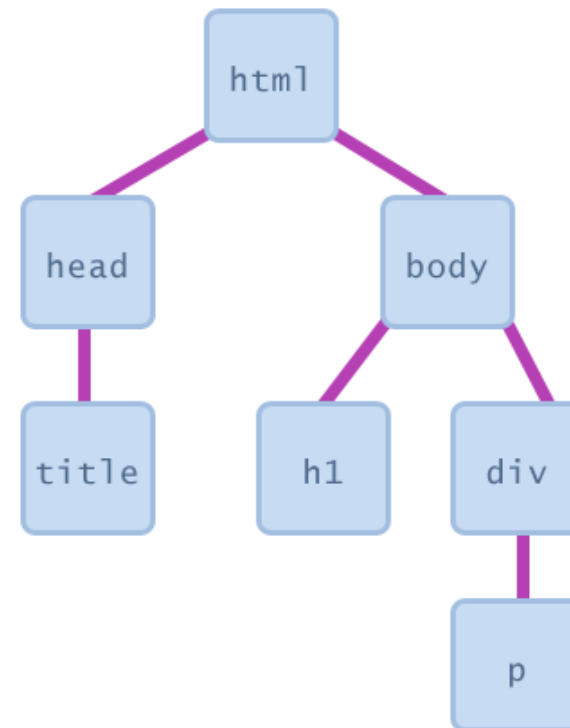
The six global DOM objects

- Every JavaScript program can refer to the following global objects

name	description
document	current HTML page and its content
history	list of pages the user has visited
location	URL of the current HTML page
navigator	info about the web browser you are using
screen	info about the screen area occupied by the browser
window	the browser window

The DOM tree

```
<html>
  <head>
    <title> ... </title>
  </head>
  <body>
    <h1> ... </h1>
    <div>
      <p> ... </p>
    </div>
  </body>
</html>
```



ข้อความธรรมดา: textContent

- textContent แสดงข้อความตามตัวอักษร โดยไม่แปลเป็น HTML
- ใช้กับข้อความจากผู้ใช้ เพื่อลดความเสี่ยง Cross-Site Scripting (XSS) จากการแทรก HTML

```
160 function addWithTextContent() {  
161     const result = document.getElementById("textContentResult");  
162     const p = document.createElement("p");  
163     p.textContent = "<strong>Hello</strong> (แสดงแท็กเป็นข้อความธรรมดา ปลอดภัยจาก XSS)";  
164     result.appendChild(p);  
165 }  
166  
167 function addWithInnerHTML() {  
168     const result = document.getElementById("textContentResult");  
169     const p = document.createElement("p");  
170     p.innerHTML = "<strong>Hello</strong> (แสดงเป็นตัวหนา แปลงเป็น HTML Tag)";  
171     result.appendChild(p);  
172 }
```

innerHTML แปลแท็กเป็น HTML จึงต้องระวังแหล่งที่มาของข้อมูล

textContent vs innerHTML — ผลการรัน

2. textContent vs innerHTML

เพิ่มด้วย textContent

เพิ่มด้วย InnerHTML

Clear

`<strong>Hello</strong>` (แสดงแท็กเป็นข้อความธรรมดา ปลอดภัยจาก XSS)

Hello (แสดงเป็นตัวหนา แปลงเป็น HTML Tag)

ข้อความเดียวกัน: textContent แสดงแท็ก ส่วน innerHTML แสดงตัวหนา

textContent vs innerHTML

- เลือกตามชนิดของเนื้อหาที่ต้องการแสดง

```
p.textContent = "<strong>Hello</strong>";
```

// แสดงแท็ก เป็นข้อความ

```
p.innerHTML = "<strong>Hello</strong>";
```

// แสดง Hello เป็นตัวหนา

- ห้ามนำข้อมูลที่ไม่น่าเชื่อถือใส่ innerHTML โดยตรง
- หากจำเป็นต้องแทรกโครงสร้าง HTML จริงๆ ต้องนำข้อมูลไปผ่านไลบรารีสำหรับกรองแท็กและคำสั่งอันตรายออกก่อน (เช่น DOMPurify)

การสร้าง nodes หรือ elements ใหม่

- เราจะทำการสร้าง node ใหม่ ก่อนที่เราจะเพิ่ม node นั้นลงใน document
- หลังจากสร้างแล้ว เราก็ต้อง add ลงไปใน document ด้วย

name	description
<code>document.createElement("tag")</code>	สร้างและสร้างคืน โหนด DOM (DOM node) ใหม่ที่ว่างเปล่า ซึ่งเป็นตัวแทนของ Element ตามชนิดที่ระบุ
<code>document.createTextNode("text")</code>	สร้างและสร้างคืน โหนดข้อความ (Text node) ที่บรรจุข้อความตามที่กำหนดไว้

การเพิ่ม/ลด nodes ใน document

- เราสามารถเพิ่ม/ลดโดยใช้ method ต่อไปนี้

name	description
appendChild(node)	นำโหนดที่กำหนดไปวางต่อท้ายรายการโหนดลูกของโหนดนี้ (เพิ่มเข้าไปเป็นลูกคนสุดท้าย)
insertBefore(new,old)	นำโหนดใหม่ที่กำหนดไปแทรกในรายการโหนดลูกของโหนดนี้ โดยวางไว้ข้างหน้าโหนดลูกเดิมที่ระบุ
removeChild(node)	ลบโหนดที่กำหนดออกจากรายการโหนดลูกของโหนดนี้
replaceChild(new,old)	แทนที่โหนดลูกเดิมที่ระบุด้วยโหนดใหม่ที่กำหนดเข้ามา

Example: การสร้าง paragraph

สร้าง element แล้วกำหนดข้อความ ก่อนเพิ่มลงใน document

```
function addParagraph() {  
  const p = document.createElement("p");  
  p.textContent = "A paragraph!";  
  document.getElementById("main").appendChild(p);  
}
```

ใช้ textContent เมื่อเนื้อหาเป็นข้อความธรรมดา

Create Element และกำหนด Style

- Style object ทำหน้าที่เป็นตัวแทนของข้อความหรือคำสั่งกำหนดสไตล์ (CSS Style Statement) แต่ละคำสั่ง
- การเข้าถึง Style object ของ Element ที่ระบุ:

```
178 function addParagraph() {  
179     const main = document.getElementById("main");  
180     const p = document.createElement("p");  
181     p.textContent = "A paragraph!";  
182     main.appendChild(p);  
183 }  
184  
185 function applyDomStyle() {  
186     const main = document.getElementById("main");  
187     main.style.color = "red";  
188     main.style.marginLeft = "50px";  
189 }  
190
```

Create Element และกำหนด Style — ผลการรัน

3. Create New Elements & Style by DOM Object

สร้าง `<p>` ใหม่ (appendChild)

กำหนด Style (color: red, marginLeft: 50px)

A paragraph!

กดสร้าง paragraph แล้วกำหนดสีแดงและระยะขอบซ้าย

Selecting groups of DOM objects

methods in document and other DOM objects (* = HTML5):

name	description
getElementsByTagName	คืนค่าเป็นอาร์เรย์ (Array/HTMLCollection) ของ element ทั้งหมดที่มีชื่อแท็กตรงตามที่ระบุ เช่น "div"
getElementsByName	คืนค่าเป็นอาร์เรย์ (NodeList) ของ element ที่มีแอตทริบิวต์ name ตรงตามที่ระบุ (ส่วนใหญ่มีประโยชน์มากสำหรับการเข้าถึงตัวควบคุมในฟอร์ม เช่น ช่องกรอกข้อมูล)
querySelector*	คืนค่าเป็น element แรกสุดที่ตรงกับเงื่อนไขของข้อความคัดเลือก CSS (CSS Selector String) ที่ระบุ
querySelectorAll*	คืนค่าเป็นอาร์เรย์ (NodeList) ของทุกๆ element ที่ตรงกับเงื่อนไขของข้อความคัดเลือก CSS (CSS Selector String) ที่ระบุ

ตัวอย่าง: Selecting DOM objects

```
195 function highlightAddressParagraphs() {  
196     const addrParas = document.querySelectorAll("#address p");  
197     for (let i = 0; i < addrParas.length; i++) {  
198         addrParas[i].style.backgroundColor = "yellow";  
199     }  
200 }  
201  
202 function resetAddressParagraphs() {  
203     const addrParas = document.querySelectorAll("#address p");  
204     for (let i = 0; i < addrParas.length; i++) {  
205         addrParas[i].style.backgroundColor = "";  
206     }  
207 }  
208
```

querySelectorAll("#address p") เลือก paragraph ภายใน #address

Selecting DOM objects — ผลการรัน

4. Selecting Groups of DOM Objects (querySelectorAll)

Highlight #address p

Reset

This is my address

1234 Street

Atlanta, GA

`querySelectorAll("#address p")` เลือก paragraph ภายใน #address

เลือก P ทุกตัว

```
212 function highlightAllParagraphs() {  
213     const allParas = document.querySelectorAll("p");  
214     for (let i = 0; i < allParas.length; i++) {  
215         allParas[i].style.backgroundColor = "lightblue";  
216     }  
217 }
```

4. Selecting Groups of DOM Objects (querySelectorAll)

Highlight #address p

Highlight p tag

Reset

This is my address

1234 Street

Atlanta, GA

Getting/setting CSS classes

- JS DOM's `className` property ใช้สำหรับ `get` หรือ `set` CSS class
- JS DOM's `classList` property ใช้สำหรับ `check` `add` `remove` classes
- `classList` เหมือนกับ `className` DOM property, แต่ `classList` สามารถใส่ class ได้หลายตัวโดยไม่ต้องกำหนดให้ แต่ละ class ทำการเว้นวรรค

classList property

Method	Description
add(class1, class2, ...)	เพิ่มชื่อคลาส ตั้งแต่หนึ่งชื่อขึ้นไปให้กับ Element (หากมีคลาสนั้นอยู่แล้วจะไม่เพิ่มซ้ำ)
contains(class)	ตรวจสอบการมีอยู่ของคลาส โดยจะคืนค่าเป็นบูลีน (true/false) เพื่อระบุว่า Element มีคลาสที่ระบุหรือไม่
remove(class1, class2, ...)	ลบชื่อคลาส ตั้งแต่หนึ่งชื่อขึ้นไปออกจาก Element (หากไม่มีคลาสนั้นอยู่ ระบบจะไม่แสดงข้อผิดพลาด)
toggle(class, true false)	สลับสถานะของคลาส (ถ้ามีคลาสอยู่แล้วจะลบออก แต่ถ้ายังไม่มีจะเพิ่มเข้าไป)

CSS classes: toggle, add, reset

```
213 function toggleHighlight() {  
214     const target = document.getElementById("classTarget");  
215     target.classList.toggle("highlight");  
216 }  
217  
218 function addHighlight2() {  
219     const target = document.getElementById("classTarget");  
220     target.classList.add("highlight2");  
221 }  
222  
223 function resetClasses() {  
224     const target = document.getElementById("classTarget");  
225     target.className = "class-target-box";  
226 }
```

classList เปลี่ยน class ส่วน className กำหนดชุด class ใหม่

CSS classes: toggle, add, reset — ผลการรัน

5. CSS Classes (className & classList)

`classList.toggle("highlight")`

`classList.add("highlight2")`

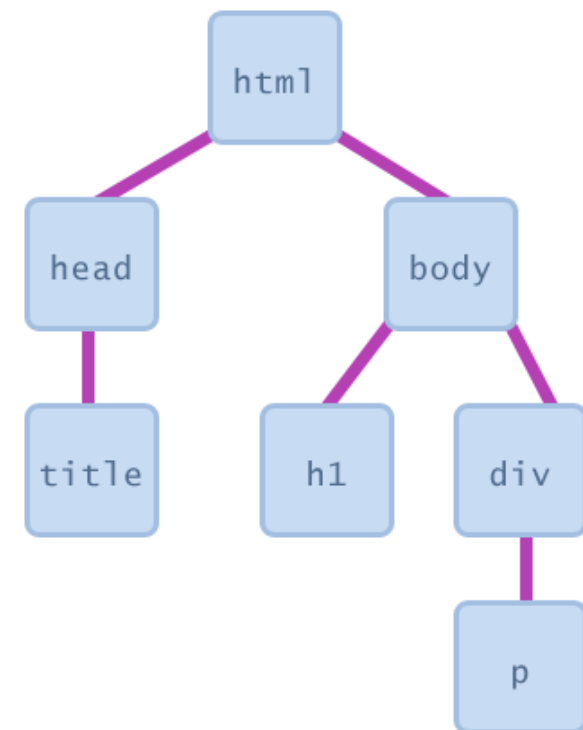
Reset Class

A paragraph!

classList เปลี่ยน class ส่วน className กำหนดชุด class ใหม่

จาก The DOM tree

```
<html>
  <head>
    <title> ... </title>
  </head>
  <body>
    <h1> ... </h1>
    <div>
      <p> ... </p>
    </div>
  </body>
</html>
```



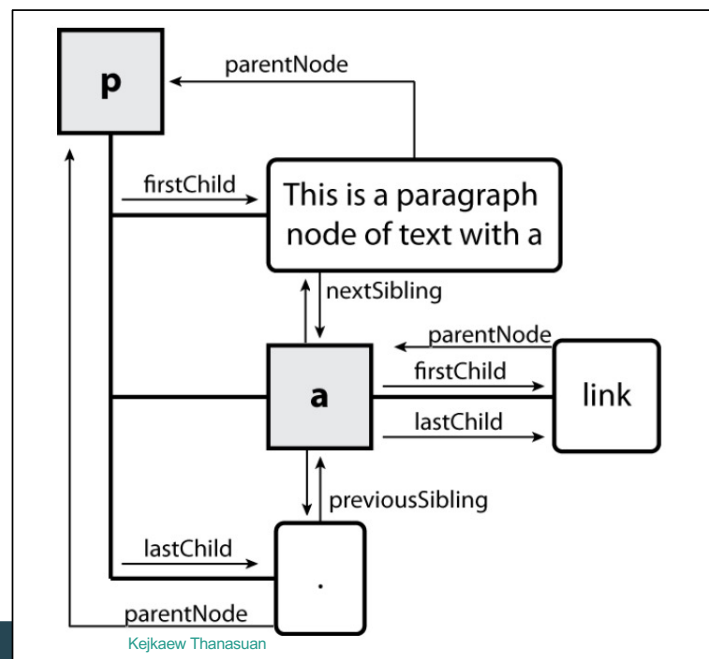
Traversing the DOM tree manually

ทุก node ของ DOM object จะมี properties ต่อไปนี้ สำหรับอ้างอิงถึง node อื่นๆ

name(s)	description
firstChild, lastChild	จุดเริ่มต้น / จุดสิ้นสุด ของรายการ โหนดลูกทั้งหมดของโหนดนี้ (โหนดลูกตัวแรกสุด / โหนดลูกตัวท้ายสุด)
childNodes	อาร์เรย์ (หรือคอลเลกชัน) ที่รวบรวมโหนดลูกทั้งหมดของโหนดนี้
nextSibling, previousSibling	โหนดข้างเคียงที่อยู่ติดกันและมีโหนดแม่ (Parent) เดียวกัน (โหนดพี่น้องข้างเคียง)
parentNode	Element ที่ครอบหรือบรรจุโหนดนี้อยู่ (โหนดแม่ / องค์ประกอบระดับพ่อแม่)

ตัวอย่าง DOM tree properties

`<p id="foo">This is a paragraph of text with a
link.</p>`



Removing a node: remove()

Modern JavaScript ลบ element ได้โดยตรง

```
234 function removeUsingRemove() {  
235     const item1 = document.getElementById("box-remove");  
236     if (item1) item1.remove();  
237 }  
238  
239 // วิธีที่ 2: Legacy JS - x.parentNode.removeChild(x)  
240 function removeUsingParentNode() {  
241     const item2 = document.getElementById("box-removeChild");  
242     if (item2 && item2.parentNode) {  
243         item2.parentNode.removeChild(item2);  
244     }  
245 }  
246
```

วิธีเดิม: x.parentNode.removeChild(x);
กดลบ Item 1 ด้วย remove() และเทียบกับวิธีเดิมของ Item 2

Removing a node: remove() — ผลการรัน

6. Removing a Node (remove vs parentNode.removeChild)

ลบด้วย `element.remove()` ลบด้วย `parentNode.removeChild()` Reset Items

Item 2 (`parentNode.removeChild()`)

ตรวจ element ก่อนลบหรือสร้างใหม่

Toggle / `removeMe()`

กดลบ Item 1 ด้วย `remove()` และเทียบกับวิธีเดิมของ Item 2

ตรวจ element ก่อนลบหรือสร้างใหม่

```
266 function toggleRemoveMe() {
267     const x = document.getElementById("removeMe");
268     if (x) {
269         x.remove();
270     } else {
271         const container = document.getElementById("mainToggleRemove");
272         const h3 = document.createElement("h3");
273         h3.id = "removeMe";
274         h3.className = "removable-item";
275         h3.textContent = "Hello World,";
276         container.appendChild(h3);
277     }
278 }
279
```

กด Toggle ซ้ำ: ลบเมื่อพบ element และสร้างใหม่เมื่อหาไม่พบ

ตรวจ element ก่อนลบหรือสร้างใหม่ — ผลการรัน

6. Removing a Node (remove vs parentNode.removeChild)

ลบด้วย `element.remove()` ลบด้วย `parentNode.removeChild()` Reset Items

Item 2 (`parentNode.removeChild()`)

ตรวจ element ก่อนลบหรือสร้างใหม่

Toggle / RemoveMe()

Hello World,

กด Toggle ซ้ำ: ลบเมื่อพบ element และสร้างใหม่เมื่อหาไม่พบ

JavaScript events (object)

- The event object

```
function name(event) {
```

```
    // an event handler function ...
```

```
}
```

property name	description
type	what kind of event, such as "click" or "mousedown"
target	the element on which the event occurred
timeStamp	when the event occurred

Mouse events

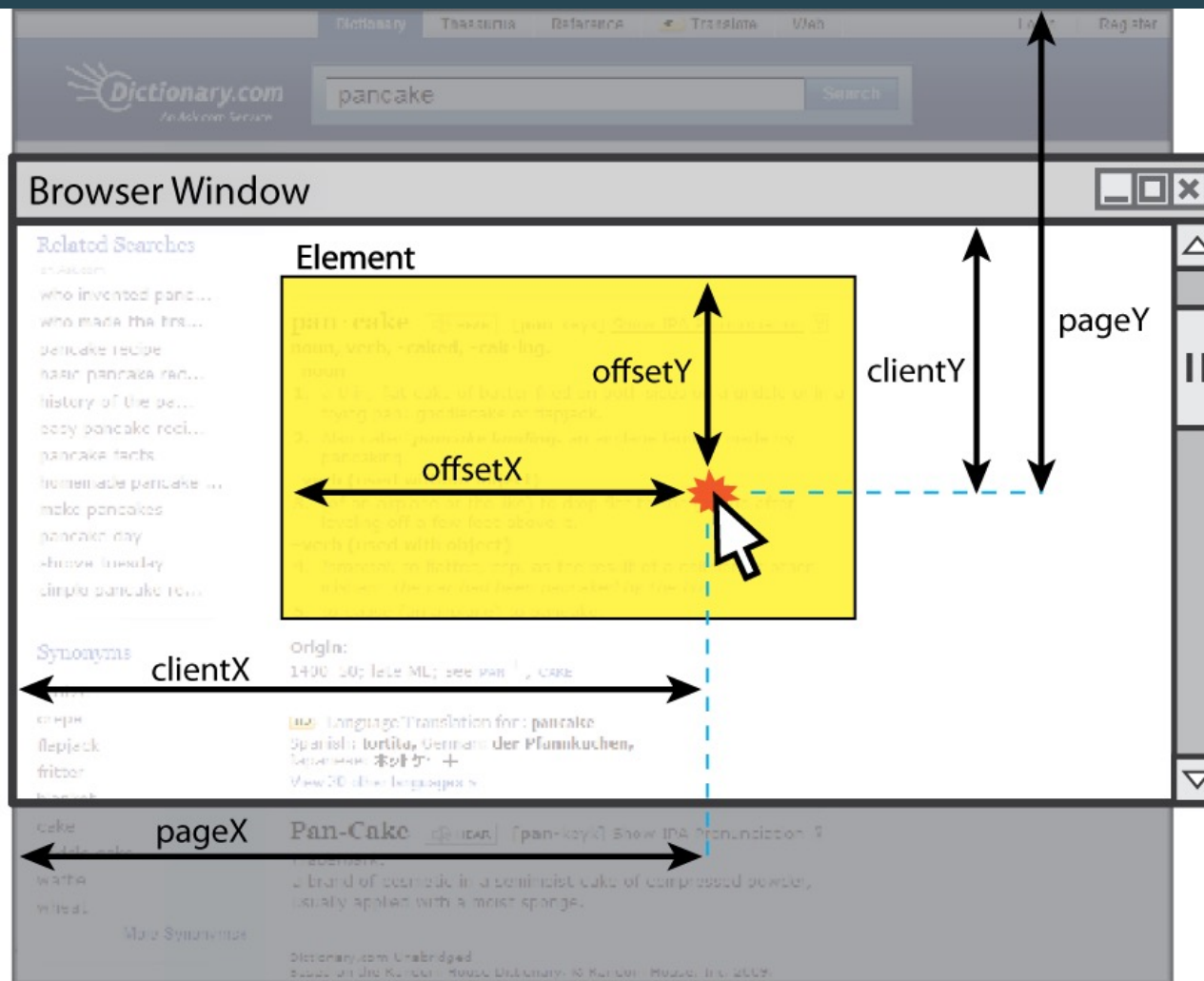
click	user presses/releases mouse button on the element
dblclick	user presses/releases mouse button twice on the element
mousedown	user presses down mouse button on the element
mouseup	user releases mouse button on the element

mouseover	mouse cursor enters the element's box
mouseout	mouse cursor exits the element's box
mousemove	mouse cursor moves around within the element's box

Mouse event objects

- The event passed to a mouse handler has these properties:

property/method	description
clientX, clientY	coordinates in <i>browser window</i>
screenX,screenY	coordinates in <i>screen</i>
offsetX, offsetY	coordinates in <i>element</i> (non-standard)
pageX, pageY	coordinates in <i>webpage</i>
button	integer representing which button was pressed (0=Left, 1=Middle, 2=Right)



Event Object: Mouse Coordinates

คลิกพื้นที่สามิตเพื่อดู screen, client และ button

```
284 function showCoords(event) {  
285     const target = document.getElementById("target");  
286     target.textContent =  
287     "screen : (" + event.screenX + ", " + event.screenY + ")\n" +  
288     "client : (" + event.clientX + ", " + event.clientY + ")\n" +  
289     "button : " + event.button;  
290 }  
291
```

Event Object: Mouse Coordinates — ผลการรัน

7. Event Object & Mouse Events (Coordinates)

```
screen : 763, 582)
client : (729, 346)
button : 0
```

คลิกพื้นที่สีแดงเพื่อดู screen, client และ button

Event Delegation: Task List

การจัดการเหตุการณ์

- สมมติว่าเรากำลังทำหน้าเว็บ **To-Do List (Task List)** ซึ่งมีรายการงานหลายข้อ และผู้ใช้สามารถกดปุ่มลบ หรือกดติ๊กถูกเพื่อทำเครื่องหมายว่าเสร็จแล้วได้:
- หากใน Task List มี 100 รายการ เราต้องสั่งวนลูปผูกคำสั่งตรวจสอบการคลิก (addEventListener หรือ onclick) ให้กับปุ่มลบทั้ง 100 ปุ่ม
- เมื่อผู้ใช้พิมพ์งานข้อใหม่แล้วกดปุ่ม "Add Task" ตัว HTML Element ที่ถูกสร้างขึ้นมาใหม่ ภายหลัง จะไม่ติดคำสั่งคลิกไปด้วย ทำให้ปุ่มลบของรายการใหม่กดแล้วไม่ทำงาน (นอกจากจะต้องเขียนโค้ดผูกคำสั่งซ้ำให้ทุกครั้งที่เราสร้างใหม่)
- ทำให้ต้องมีการใช้ Event delegation

- แทนที่เราจะเอาคำสั่งไปดักที่ปุ่มของทุกรายการ เราจะนำคำสั่งดักจัดการคลิกไปผูกไว้ที่ "กล่องแม่ตัวเดียว" (เช่น แท็ก `<ul>` หรือ `<div id="task-list">`) แล้วใช้คุณสมบัติ `event.target` ตรวจสอบว่า สิ่งที่ถูกคลิกจริงๆ คือปุ่มใด

```
324 function handleTodoListClick(event) {  
325     // 1. ถ้าคลิกโดนปุ่มลบ  
326     if (event.target.classList.contains("delete-btn")) {  
327         const item = event.target.closest(".todo-item");  
328         if (item) {  
329             item.remove();  
330             updateTaskCount();  
331         }  
332     }  
333     // 2. ถ้าคลิกที่ตัวข้อความ หรือแถวรายการ -> สลับสถานะ completed  
334     else if (event.target.classList.contains("task-text") ||  
335             event.target.classList.contains("todo-item")) {  
336         const item = event.target.closest(".todo-item");  
337         if (item) {  
338             item.classList.toggle("completed");  
339         }  
340     }  
341 }
```

listener อยู่ที่ #todo-list จุดเดียว และใช้ `event.target` ตรวจสอบสิ่งที่คลิก

Event Delegation: Task List — ผลการรัน

8. Event Delegation (Interactive Task List)

+ เพิ่มรายการ

รายการทั้งหมด: 3 งาน

Task 2: ศึกษา Event Delegation บน Container



Task 3: ทดสอบ textContent vs innerHTML



Task 4: ทดลองเพิ่มรายการใหม่



listener อยู่ที่ #todo-list จุดเดียว และใช้ event.target ตรวจสอบสิ่งที่คลิก

Assignment 7:

- Download assignment7.zip file
- ทำการสร้าง square game โดยในเกมนี้ เราจะต้องกำหนดก่อนว่าเราจะให้มีสีเหลี่ยมทั้งหมดกี่อัน และสีของสีเหลี่ยม เมื่อเรากด Start แล้ว เราจะต้องกดลบสีเหลี่ยมทั้งหมด ภายในเวลา 30 วินาที ถึงจะชนะในเกมนี้
- การบ้าน ทำการตกแต่ง square game ของเรา ให้สวยงาม

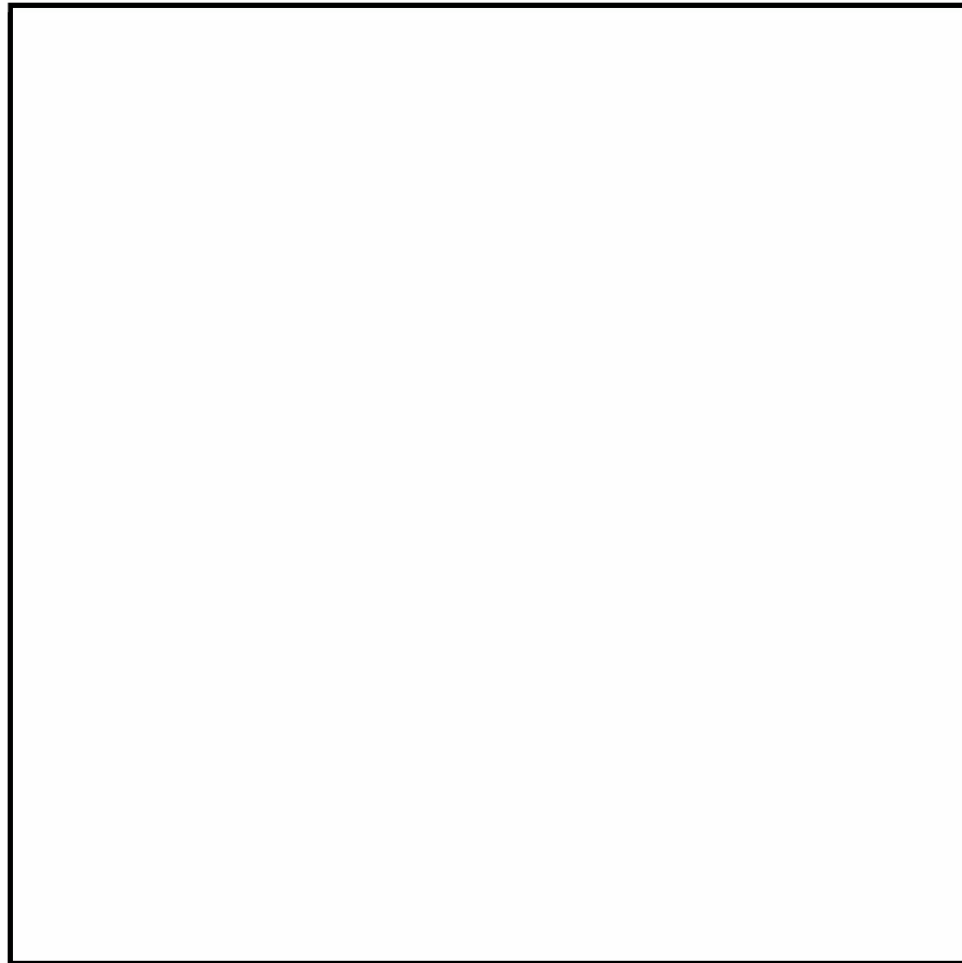
Control

Number of box:

Maroon ▼

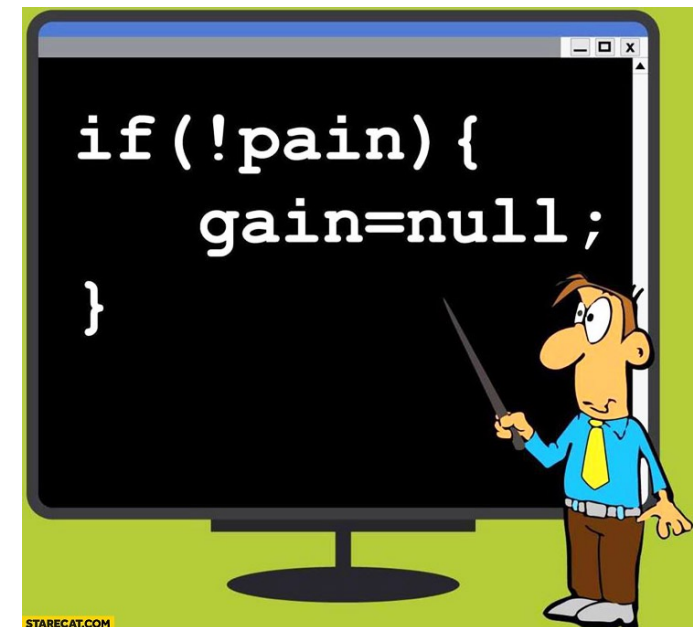
Start

Timer:



Week 7: Pop Quiz

- ตอบคำถาม 8 ข้อแบบออนไลน์
- มีเวลา 15 วินาทีในการตอบแต่ละข้อ
- <https://ketnas.github.io/popQuiz/>



References

- <https://www.w3schools.com/>
- <https://courses.cs.washington.edu/courses/cse154/17sp/>
- *Web Programming Step by Step* by Marty Stepp, Jessica Miller, and Victoria Kirst, <http://www.webstepbook.com/>